

Executable 2D RPG Visual Design Guide for Eldoria V2

Executive Summary

Because the existing **Eldoria-V2 Design Guide** was not included in this chat, the guide below is written as a **drop-in extension layer**: it establishes deterministic art, asset, animation, naming, and runtime rules that Codex and Claude can follow without guessing. The assumptions I use are a **pixel-art, fantasy 2D RPG**, primarily viewed in **3/4 top-down exploration**, targeting **PC and mobile**, with a **mobile-first performance posture**. The most stable technical baseline for that combination is a **640×360 logical world resolution**, **integer scaling where possible**, a **16×16 tile grammar**, and a **sprite-atlas workflow** rather than loose per-file texture loading. That recommendation aligns with Godot's pixel-art resolution guidance, Unity's pixel-perfect tooling, and both engines' emphasis on atlases and UI scaling controls. ¹

The core design decision in this report is to treat the guide as a **machine-readable production contract**, not just an art bible. That means every asset should have a canonical ID, pivot, logical footprint, palette family, animation tag set, layer order, and optional slice metadata for 9-slice UI or attachment points. This is directly supported by Aseprite's tag, slice, pivot, JSON export, and sprite-sheet workflows; by Unity's Aseprite importer, Sprite Atlas, and per-sprite pivot/import settings; and by Godot's TextureAtlas, CanvasLayer, Control, and collision-layer systems. ²

For visual style, the safest "single source of truth" is a **limited-palette, high-readability pixel-art style** with **consistent top-left key lighting**, deliberate silhouette separation, and restricted shading noise. Derek Yu's pixel-art guides emphasize constrained palettes, readable forms, reduced jaggies, and avoidance of too many similar colors; Slynrd's pixel-art tutorials emphasize small looping cycles, economy of motion, clear item readability, 8-direction top-down tradeoffs, and strong melee timing. Those are the right foundations for a code-generatable guide because they reduce ambiguity while keeping the style production-friendly. ³

In practical terms, this extension recommends: **PNG as the default runtime raster format**, `.aseprite` **as the preferred editable source for sprite work**, **SVG only for selected scalable UI assets**, **8-direction support as an enhanced target rather than a mandatory baseline**, **frame-by-frame animation for hero moments and combat**, **cutout/skeletal only when the project explicitly needs reuse-heavy content**, and **scene- or feature-based atlases** to control memory and draw calls. Unity's 2D workflow explicitly distinguishes frame-by-frame, cutout, and skeletal animation, and notes that frame-by-frame is more resource-intensive; both Unity and Godot document that atlasing reduces state changes and draw overhead. ⁴

Assumptions and Extension Scope

This report assumes **an exploratory fantasy RPG with combat**, where readability matters more than hyper-detailed rendering. The visual target is **stylized pixel art**, not freeform painted 2D. If Eldoria-V2 already defines a stronger stylistic identity — for example, painterly sprites, anime portraits, super-deformed characters, or a side-view combat presentation — those aesthetic decisions should override the assumptions below, while the **asset naming, metadata, export, collision, pipeline, and performance rules** can remain nearly unchanged. That separation is important because the technical rules are largely engine- and style-agnostic, while the visual assumptions are not. ⁵

The recommended **world reference resolution** is **640×360**. Godot's documentation states that most pixel-art games use viewport sizes between **256×224 and 640×480**, and specifically calls out **640×360** as a good baseline because it integer-scales cleanly to 720p, 1080p, 1440p, and 2160p. For World rendering, this guide therefore assumes **640×360 logical gameplay**, scaled up with **integer multiples** on PC and, when device aspect ratios force compromise, with **letterboxing or controlled expand behavior** on mobile. Unity's pixel-perfect camera and Godot's integer stretch guidance both support this recommendation. ⁶

The recommended **tile grammar** is **16×16 world tiles**, with entities authored against that grid. Slynryd's top-down character article explicitly notes the value of conforming character scale to a consistent tile grid even when the environment system is not strictly tile-based, because it improves cohesion between characters and level design. Based on that logic, this guide uses the following production assumptions: **character footprint = 16×16, standard actor canvas = 32×48, large actor canvas = 48×64, sub-boss canvas = 64×64 to 96×96, and boss modules = 96×96 to 192×192**, sometimes split into separate parts for reuse and culling. ⁷

For UI, the guide assumes a separate **screen-space layer** and a distinct scaling policy from the world layer. Unity's Canvas Scaler is built specifically to scale UI elements relative to a reference resolution, and Godot's CanvasLayer and Control systems are designed to keep UI fixed in screen space while the world camera moves independently. Accordingly, this guide assumes **HUD/UI reference design at 1920×1080** for layout logic, while still allowing pixel-art textures inside that UI system. ⁸

Core Style and Technical Contract

The single most useful way to make this guide “executable” for AI tooling is to define **hard rules** first, then a few permitted alternatives. The rules below are intentionally rigid so code generators, exporters, and art prompts all resolve to the same outcome.

Global style rules

Use a **limited palette** and keep the style visually constrained. Derek Yu's tutorials argue that pixel art benefits from constrained palettes and that reducing overly similar colors improves readability; Aseprite's Indexed mode exists precisely to enforce a bounded palette workflow. The practical rule here is: maintain **one master palette** for the project, with **sub-palettes** for biome/material families, and forbid per-asset ad hoc colors unless the asset is marked `palette_mode: free`. ⁹

Use a **single dominant light direction** for the entire game: **upper-left, slightly front-facing**, with shadows falling down-right/back. Derek Yu's introductory tutorial uses exactly this kind of simplified, consistent lighting assumption to create readable form. The operational rule is: all gameplay sprites, props, buildings, and monsters follow the same light direction unless a cutscene, spell effect, or boss phase explicitly overrides it. ¹⁰

Keep silhouettes and cluster work readable before adding secondary shading. Derek Yu warns that too many similar colors reduce readability, and Slynrd notes that looped animation exposes every pixel-level mistake. Therefore: prefer **solid readable volumes**, then **one shadow step**, then **one highlight step**, and only then optional material accents. Dithering is permitted only on backgrounds, skies, fog, and large environmental surfaces; it should be avoided on small gameplay-critical sprites and UI. ¹¹

Use **point/nearest sampling** for pixel-art gameplay sprites. Unity's Sprite import docs show Point filtering as the non-filtered mode, Unity's pixel-perfect docs are explicitly about keeping pixel art crisp, and Godot recommends nearest-neighbor behavior for pixel art and integer scaling to avoid distortion. ¹²

File format policy

Format	Use in this guide	Strengths	Risks	Decision
<code>.aseprite</code> / <code>.ase</code>	Primary editable source for sprites, layered equipment, frame timing, tags, slices	Native support for layers, frames, tags, pivots, palette workflows; importable in Unity; ideal for JSON and sprite-sheet export ¹³	Not a universal runtime format; Godot does not treat it as a first-class runtime texture asset	Preferred source format for sprite art
<code>.png</code>	Default runtime raster export	Broad engine support, alpha support, stable for sprite sheets and UI art ¹⁴	8-bit/channel; uncompressed runtime memory can be expensive	Default runtime export
<code>.webp</code>	Optional for large non-pixel-art UI illustrations, promo art, or large soft-background layers in Godot workflows	Transparency support; lossy or lossless modes in Godot import system ¹⁵	Can introduce artifacts; not ideal for crisp pixel-art gameplay sprites	Optional, not default

Format	Use in this guide	Strengths	Risks	Decision
<code>.svg</code>	Limited-use UI icons, scalable HUD ornaments, high-DPI menus	Resolution-independent UI scaling in Unity UI Toolkit and Godot DPITexture workflows ¹⁶	SVG feature subsets are limited in both engines; text should be converted to paths; runtime behavior varies	Use only for selected UI assets
<code>.psd</code> / <code>.psb</code>	Optional concept/UI source when non-pixel layered art is required	Familiar layered authoring pipeline; Unity supports PSD-related sprite import workflows via 2D features and PSD-specific import options ¹⁷	Heavier files, less deterministic for palette-constrained production	Secondary source format

The runtime export rule should therefore be simple: **gameplay sprites, tiles, monster sheets, armor sheets, particles, and cutscene stills export to PNG; source-of-truth sprite work stays in `.aseprite`; SVG is reserved for icons and non-gameplay UI only.** That choice maximizes compatibility across Unity and Godot while preserving Aseprite's authoring strengths. ¹⁸

Palette system policy

Palette system	How it works	Best use	Recommendation
Fixed master palette	Whole project shares one indexed palette	Cohesive retro/pixel identity, easy AI prompting, fast palette swaps	Best default for Eldoria-style consistency. Aseprite Indexed mode and Derek Yu's constrained-palette guidance strongly support this. ¹⁹
Master palette + sub-palettes	Global palette plus biome/material ramps	Richer worlds without style drift	Preferred production model. Use a project master plus named subsets like <code>forest</code> , <code>ruins</code> , <code>arcane</code> , <code>lava</code> , <code>ui_neutral</code> . ²⁰
Per-asset free palette	Each asset picks its own local colors	Painterly or illustrative projects, marketing art	Avoid for gameplay assets because readability and cohesion degrade quickly. Derek Yu explicitly warns about too many similar colors and low readability. ²¹

The practical palette rule is: **48 to 64 master colors**, then **material ramps** embedded in that master palette. A good starting split is **6 neutral UI colors, 6 skin/human variation colors, 6 metal colors, 6 wood/leather colors, 6 natural greens, 6 earth/stone colors, 6 magic blues/purples, 6 fire/warm colors**, plus a few reserved accents and transparent indices. This count is not an engine requirement; it is a guide recommendation derived from modern limited-palette practice, Aseprite Indexed-mode constraints, and Derek Yu's readability-first palette advice. ¹⁹

Animation technique policy

Technique	Engine support	Pros	Cons	Policy
Frame-by-frame	Unity explicitly supports it; Godot supports sheets or individual frames with AnimatedSprite2D/ AnimationPlayer ²²	Best style control, best for attacks, spellcasts, monsters, bosses	Most expensive to produce and maintain	Default for combat and gameplay-critical animation
Cutout	Unity supports cutout workflows in 2D character animation ²³	Reusable parts, lower redraw cost, good for capes or UI puppeting	Can look stiff in pixel art if overused	Allowed for secondary motion or dialogue puppets
Skeletal	Unity supports skeletal 2D workflows; Godot can do bone-driven pipelines by project-specific setup ²³	High reuse, scalable for large cast	Risk of style mismatch with pixel-art aesthetics	Use only when explicitly justified

Unity's own workflow documentation says frame-by-frame is relatively resource-intensive, so the right compromise is not "all skeletal" or "all frame-by-frame," but rather **frame-by-frame for key gameplay actions** and **reuse-driven methods only where the style tolerates them**. For Eldoria-style production, hero locomotion, attacks, hurt, cast, death, boss telegraphs, and final-hit VFX should remain frame-by-frame. ²³

Naming, metadata, and folder contract

Use **lowercase snake_case**, never spaces, never mixed separators, and never engine-generated default names like `Sprite 12` or `New Sprite`. Unity's Aseprite importer tracks layer/sprite data by name/path unless UUIDs are used, so deterministic naming matters for import stability as well as automation. ²⁴

Use this canonical asset ID shape:

```
<domain>_<entity>_<variant>_<view>_<state>_<part>_<frame>_v###
```

Examples:

```
char_knight_m_front_idle_body_f00_v001
char_knight_m_front_walk_body_f03_v001
armor_knight_steel_front_walk_torso_f03_v001
mob_wolf_gray_left_attack_body_f04_v002
ui_inventory_panel_idle_base_v001
fx_fireball_small_loop_core_f05_v001
```

```
env_ruins_wall_a_top_v003
cut_intro_hero_closeup_talk_mouth_f01_v001
```

Use these domain prefixes only:

```
char npc armor weapon mob boss env bld ui fx cut port icon tile sfx
```

Use this machine-readable metadata sidecar for every exported sheet or loose sprite:

```
{
  "id": "char_knight_m_front_walk",
  "category": "char",
  "canvasPx": [32, 48],
  "footprintPx": [16, 16],
  "pivotPx": [16, 6],
  "palette": "master_human_knight",
  "ppu": 16,
  "view": "front",
  "loop": true,
  "tags": ["walk", "front", "base_body"],
  "slices": ["weapon_socket_main", "ui_nineslice_optional"],
  "collision": {
    "hurtbox": [10, 4, 12, 18],
    "hitboxes": []
  }
}
```

Aseprite already supports **tags**, **slices**, **pivot data**, and **JSON sprite-sheet metadata**, so this sidecar format is not speculative; it is an intentionally normalized layer over Aseprite's native export vocabulary. ²⁵

Resolution, import, and atlas policy

Use **Pixels Per Unit = 16** for world assets by default. Unity's sprite import settings define PPU as the mapping between texture pixels and world units; choosing 16 makes the logical tile size and the engine-space grid line up cleanly. For UI sprites, keep `Reference Pixels Per Unit` aligned to the chosen UI convention so 1 authored UI pixel behaves predictably in Canvas scaling. ²⁶

Import gameplay pixel-art sprites with **Point filtering**, **no read/write unless absolutely necessary**, and **no mipmaps for most gameplay-facing sheets**. Unity explicitly warns that Read/Write doubles texture memory, and Godot recommends **Lossless** as the common and recommended setting for 2D pixel art while warning that VRAM-compressed modes produce visible artifacts on 2D/pixel-art assets. ²⁷

Pack art into **scene- or feature-based atlases**, not a single mega-atlas. Unity recommends separate atlases for sprites used at the same time, for content that needs different compression, and for frequently versus

infrequently used assets. Godot’s TextureAtlas import mode likewise exists to reduce memory usage for animated 2D sprites. The guide rule is: one atlas family for `characters`, one for `combat_fx`, one per biome/scene for `environment`, and one or more for `ui`. ²⁸

For atlas sizing, prefer **2048×2048 as the default cap** and allow **4096×4096 only for PC or high-end mobile after profiling**. Godot notes that large textures can create issues on mobile/web and explicitly references **4096×4096** as a practical upper concern for display support. Unity’s texture-compression docs further reinforce that platform support and memory behavior differ substantially across mobile devices, especially Android. ²⁹

Animation frame counts and timing policy

Use **per-frame timing**, not a single global FPS assumption. Aseprite supports tagged animations and Unity’s Aseprite importer supports **individual frame timings**, which makes it possible to encode anticipation, contact, overshoot, and recovery directly in the source file. ³⁰

Animation	Frames	Timing target	Notes
Idle	4–6	4–8 fps equivalent, with holds	Small breathing, cloth, or blink motion only; avoid noisy loops. Slynrd recommends simple small loops for starters and warns loop detail is highly visible. ³¹
Walk	6–8	8–12 fps	Standard exploration default
Run	8	12–16 fps	Use only if game has meaningful run state
Light attack	6–8	10–14 fps, uneven frame holds	Must include anticipation, contact, recovery; Slynrd’s melee note on overshoot is especially useful here. ³²
Heavy attack	8–12	8–12 fps	Longer anticipation and recovery
Cast	6–10	8–12 fps	Pair with separate VFX sheet
Hurt	2–4	10–14 fps	Should snap, not linger
Death	8–12	8–12 fps, non-looping	End on a final held frame
Item idle	4–6	4–8 fps	Slynrd recommends consistent item idle systems like bob, shine, or bounce. ³³

The standard sprite-sheet layout for character sheets should be **one animation tag per row, frames advancing left-to-right**, because that is the least ambiguous format for both human teams and code generators. Aseprite supports horizontal, vertical, and matrix exports, but row-separated matrices are easier to parse and to diff in source review. ³⁴

Recommended row order:

```
idle, walk, run, attack_light, attack_heavy, cast, hurt, death, special
```

Recommended directional ordering inside a set:

```
front, back, left, right, front_left, front_right, back_left, back_right
```

If the production must stay lean, support **4-direction as the minimum** and upgrade to **8-direction** only for heroes, bosses, and combat-critical enemies. Slynryd specifically notes that 4-direction is still a strong “bang for buck” design choice in top-down games, while full 8-direction support demands more from combat and level design. ⁷

Collision, layering, particles, and performance policy

Treat **hurtboxes**, **hitboxes**, and **interaction triggers** as separate authored concepts. Godot distinguishes collision layers and masks for structured interactions, and Unity uses layer-based collision detection plus trigger-collider rules that require at least one Rigidbody in the pair. The guide rule is: every combat-capable actor gets exactly one canonical hurtbox root, zero or more transient hitboxes, and optional interaction triggers on separate layers. ³⁵

Use this collision layer map:

Layer	Purpose
<code>world_solid</code>	terrain, walls, collision tiles
<code>actor_body</code>	player and enemy locomotion bodies
<code>hurtbox</code>	damage receivers
<code>hitbox_player</code>	player attacks
<code>hitbox_enemy</code>	enemy attacks
<code>proj_player</code>	player projectiles
<code>proj_enemy</code>	enemy projectiles
<code>pickup_trigger</code>	items and loot
<code>interact_trigger</code>	doors, NPC talk zones
<code>ui_world</code>	diegetic world-space UI only

For render order, use **sorting layers / z groups first**, then intra-group offsets. Unity exposes Sorting Layer and Order in Layer, while Godot uses `CanvasLayer`, `z_index`, and optional Y-sorting behavior. The guide standard is:


```
background -> terrain -> decals_low -> actors_feet -> actors_body -> actors_head
-> armor_overlays -> weapons_front -> vfx_low -> vfx_high -> weather -> world_ui
-> screen_ui
```

That aligns with Unity's 2D sorting model and Godot's `CanvasLayer` / `z_index` model. ³⁶

For particles and VFX, prefer **flipbook particles for complex effects** and **small procedural particle bursts for low-cost feedback**. Unity's Texture Sheet Animation module and Godot's GPUParticles2D/ParticleProcessMaterial systems both support flipbook-style animated particle textures. Use particles for sparks, dust, embers, healing motes, slash residue, projectile trails, rain splashes, and telegraph rings; do not use them to replace all key combat animation. ³⁷

Recommended VFX sheet sizes are **16×16**, **32×32**, and **64×64** cells only. Keep VFX authored in named families such as `impact_physical`, `impact_magic_arcane`, `trail_fire_small`, `telegraph_cone_red`, `status_poison_burst`, and `weather_rain_drop`. Each family gets one master tint ramp and one additive variant. This is a guide standard rather than an engine rule, but it is consistent with the flipbook- and atlas-driven production model documented by Unity, Godot, and Aseprite. ³⁸

The following are recommended **mobile-first starting budgets** for a typical on-screen gameplay scene. They are not engine-imposed limits; they are synthesis targets derived from the documented cost of texture memory, read/write duplication, draw calls, batching/atlasing, and mobile compression behavior. ³⁹

Budget item	Mobile target	PC target	Why
Visible draw calls	≤ 80 exploration, ≤ 120 battle	≤ 150 exploration, ≤ 220 battle	Atlasing and batching matter in both engines. ⁴⁰
Resident 2D texture memory per scene	≤ 96 MB low-end / ≤ 128 MB mid-tier	≤ 256 MB	Unity and Godot both frame texture memory as a major performance variable; read/write duplication and incompatible compression both hurt. ⁴¹
Atlas count simultaneously resident	≤ 6 gameplay atlases + 1 UI atlas	≤ 10 gameplay atlases + 2 UI atlases	Fewer texture swaps, simpler residency management. ⁴²
Per-character loaded animation memory	≤ 2–4 MB compressed or equivalent	≤ 8 MB	Prevents cast explosion on mobile
Live particles on screen	≤ 120 small / ≤ 24 large flipbook emitters	≤ 300 small / ≤ 60 large	Godot notes GPU requirements rise with particle amount; same design logic applies in Unity. ⁴³

Category Rules

The table below is the concise, AI-usable rule set for each requested art category.

Category	Visual rule	Technical rule	Animation rule	Notes
Sprites	Heroic but readable silhouettes; 3/4 top-down projection; chunky limbs, no 1-pixel gameplay-critical anatomy	Base canvas 32×48 , footprint 16×16 , pivot at center-bottom footprint	Idle 4–6, walk 6–8, attack 6–8 minimum	Readability and chunky forms follow Derek Yu; top-down tile coherence follows Slynrd. ⁴⁴
Armor	Armor must inherit body canvas and pivot exactly; no independent drift	Separate layers: <code>body</code> , <code>torso</code> , <code>legs</code> , <code>pauldrons</code> , <code>gloves</code> , <code>helm</code> , <code>cape</code> , <code>weapon_back</code> , <code>weapon_front</code>	Match base-body timing 1:1	In Unity, layered Aseprite import preserves order through Order in Layer and Sorting Group. ²⁴
Combat animations	Emphasize anticipation, contact, overshoot, recovery	Hitbox frames authored explicitly in metadata; no guessed timing in code	Light 6–8, heavy 8–12, cast 6–10, hurt 2–4	Per-frame timing support exists in Aseprite/Unity; melee overshoot principle from Slynrd. ⁴⁵
Monsters and bosses	One readable silhouette, one dominant material story, one accent hue	Standard mobs 32×32 to 64×64; bosses split into modules when >96×96	Telegraphs must be readable before contact frames	Use stronger contrast than player characters so enemies read instantly. ²¹
UI	Use 9-slice panels, iconography, and high-contrast states	Screen-space UI layer; 1920×1080 layout reference; vector only where scale helps	UI motion is subtle and short	Unity <code>CanvasScaler</code> , Godot <code>Control</code> , <code>CanvasLayer</code> , and <code>NinePatchRect</code> back this model. ⁴⁶

Category	Visual rule	Technical rule	Animation rule	Notes
Landscapes	Prioritize large readable masses over micro-detail	Tiles 16×16 or 32×32 multiples; autotiles optional; atlas by biome	Ambient loop only when it adds gameplay value	Background detail should not compete with actors. Derek Yu's readability argument applies strongly here. ²¹
Buildings	Same projection and light direction as world props; entrances strongly marked	Block from tile modules first, add trim layers second	Very limited motion, mostly flags/signs/windows	Use 9-slice or tile repeats where scaling is needed. ⁴⁷
NPCs	Reuse player body language grammar; distinct costume accents and props	Same body rig/canvas as playable humans unless lore requires otherwise	Idle, walk, emote, talk; combat only if needed	Shared rigs reduce production cost and improve cohesion. ⁴⁸
Cutscenes	Reuse gameplay palettes and light rules; allow portrait inserts and overlay FX	Separate ^{cut} domain, but keep naming and pivots consistent	Camera pans, fades, portrait swaps, mouth flaps, VFX overlays	Use CanvasLayer/UI overlay systems for transitions and dialogue framing. ⁴⁹

For **UI typography and iconography**, keep the core HUD art low-noise and readability-first. If SVG is used, use it only for ornamental or scalable interface elements, not for text or gameplay sprites; both Unity and Godot document limitations in SVG support and recommend careful import choices. Panels should be authored as **9-slice assets** using either Aseprite slices or engine-native 9-slice/NinePatch systems. ⁵⁰

For **LOD behavior in 2D**, the most important LOD is usually not mesh complexity but **texture residency and scene visibility**. Use full-detail hero sheets near the camera, reduced atlas variants or reduced animation density for far/background actors, and static impostor or loop-reduced variants for crowd/background NPCs. Unity documents atlas variants and lower-resolution atlas workflows, and general mobile optimization guidance explicitly calls out reducing texture size and using LOD-style simplification. ⁵¹

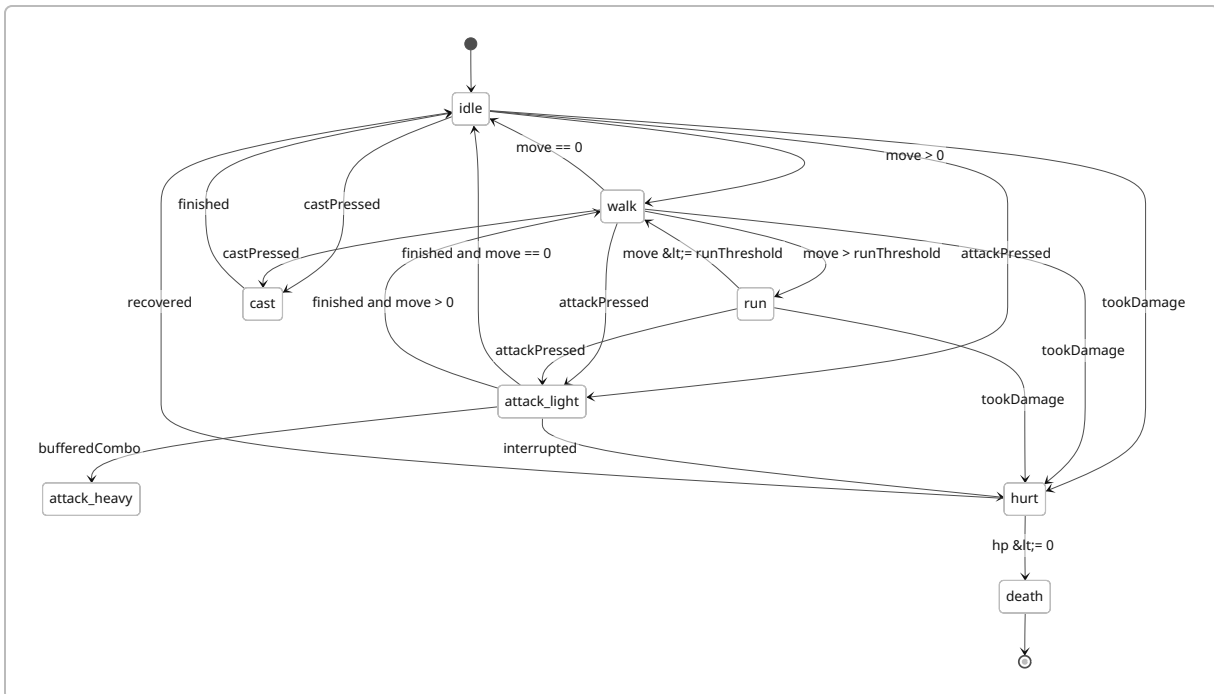
Runtime Integration and Asset Pipeline

The recommended asset pipeline is:



This pipeline mirrors Aseprite's source-first model, JSON/slice export capabilities, Unity's Aseprite importer and Sprite Atlas workflow, and Godot's texture-atlas and image-import pipeline. 52

The recommended animation state contract is:



This state machine is intentionally modest. It avoids overfitting to a single engine and preserves the support for per-frame timing, non-looping tags, and event-bearing import workflows documented by Aseprite and Unity. 53

Recommended folder structure:

```
/assets
  /art
    /characters
      /knight_m
        /src
          knight_m.aseprite
        /export
          char_knight_m_front_idle.png
          char_knight_m_front_walk.png
          char_knight_m_front_attack_light.png
        /meta
          char_knight_m.asset.json
      /armor
        /knight_steel
          /src
```

```
    /export
    /meta
  /monsters
  /bosses
  /environment
    /forest
    /ruins
    /village
  /buildings
  /ui
    /hud
    /menus
    /icons
  /fx
    /combat
    /magic
    /weather
  /cutscenes
  /atlases
    /gameplay
    /biomes
    /ui
  /manifests
    asset_registry.json
    palette_registry.json
    animation_registry.json
```

A folder structure like this works especially well with the documented Aseprite source/export split, Unity's atlas workflow, and Godot's texture-atlas/import pipeline. It is also easy for LLMs to reason about because each asset has one obvious home. ⁵⁴

Pseudocode

```
for each asset_source in /src:
    validate_filename(asset_source)
    validate_palette(asset_source)
    validate_canvas_and_pivot(asset_source)
    export_png_sheet(asset_source)
    export_json_metadata(asset_source)

group exports by atlas_family
pack_atlas(atlas_family)

for each runtime_entity:
    load atlas + metadata
    bind clips by tag name
```

```
bind hurtbox + hitbox frames
bind sorting layer + pivot + footprint
register ui icon if present
```

That pseudocode reflects the tag-, atlas-, and metadata-first pipelines supported by Aseprite, Unity, and Godot. 55

Engine-agnostic JS and TS example

```
type Rect = { x: number; y: number; w: number; h: number };

type FrameDef = {
  rect: Rect;
  durationMs: number;
  hurtbox?: Rect;
  hitbox?: Rect;
};

type ClipDef = {
  name: string;
  loop: boolean;
  frames: FrameDef[];
};

type SpriteAsset = {
  id: string;
  image: HTMLImageElement;
  pivotPx: [number, number];
  footprintPx: [number, number];
  clips: Record<string, ClipDef>;
};

class SpritePlayer {
  private clip!: ClipDef;
  private frameIndex = 0;
  private timerMs = 0;

  constructor(public asset: SpriteAsset) {}

  play(name: string, reset = false) {
    const next = this.asset.clips[name];
    if (!next) throw new Error(`Missing clip: ${name}`);
    if (reset || this.clip !== next) {
      this.clip = next;
      this.frameIndex = 0;
      this.timerMs = 0;
    }
  }
}
```

```

    }
}

update(dtMs: number) {
    if (!this.clip) return;
    this.timerMs += dtMs;

    while (this.timerMs >= this.clip.frames[this.frameIndex].durationMs) {
        this.timerMs -= this.clip.frames[this.frameIndex].durationMs;
        this.frameIndex++;

        if (this.frameIndex >= this.clip.frames.length) {
            this.frameIndex = this.clip.loop ? 0 : this.clip.frames.length - 1;
        }
    }
}

draw(ctx: CanvasRenderingContext2D, worldX: number, worldY: number) {
    const f = this.clip.frames[this.frameIndex];
    const [pivotX, pivotY] = this.asset.pivotPx;
    ctx.imageSmoothingEnabled = false;
    ctx.drawImage(
        this.asset.image,
        f.rect.x, f.rect.y, f.rect.w, f.rect.h,
        Math.round(worldX - pivotX),
        Math.round(worldY - pivotY),
        f.rect.w, f.rect.h
    );
}

getCurrentHurtbox(worldX: number, worldY: number): Rect | null {
    const f = this.clip.frames[this.frameIndex];
    if (!f.hurtbox) return null;
    return { ...f.hurtbox, x: worldX + f.hurtbox.x, y: worldY + f.hurtbox.y };
}

getCurrentHitbox(worldX: number, worldY: number): Rect | null {
    const f = this.clip.frames[this.frameIndex];
    if (!f.hitbox) return null;
    return { ...f.hitbox, x: worldX + f.hitbox.x, y: worldY + f.hitbox.y };
}

function overlaps(a: Rect | null, b: Rect | null): boolean {
    if (!a || !b) return false;
    return a.x < b.x + b.w && a.x + a.w > b.x && a.y < b.y + b.h && a.y + a.h >
    b.y;
}

```

```

class HealthBarUI {
    constructor(private element: HTMLDivElement) {}
    setNormalized(value: number) {
        this.element.style.width = `${Math.max(0, Math.min(1, value)) * 100}%`;
    }
}

```

This example intentionally encodes the guide's key rules: **nearest/point rendering**, **per-frame timing**, **metadata-defined pivots**, and **frame-authored hitbox/hurtbox logic**. Those are the same concepts formalized by Aseprite tags/slices, Unity sprite imports, and engine collision/UI systems. 56

Unity C# example

```

using UnityEngine;
using UnityEngine.UI;
using UnityEngine.U2D;

public class EldoriaActor2D : MonoBehaviour
{
    [Header("Rendering")]
    [SerializeField] private SpriteAtlas atlas;
    [SerializeField] private SpriteRenderer spriteRenderer;
    [SerializeField] private string[] walkFrameNames;
    [SerializeField] private float[] walkFrameDurations = { 0.10f, 0.08f,
0.10f, 0.08f };

    [Header("Combat")]
    [SerializeField] private Collider2D hurtbox;
    [SerializeField] private Collider2D hitbox; // child trigger, normally
disabled
    [SerializeField] private int maxHp = 100;

    [Header("UI")]
    [SerializeField] private Image healthFill;

    private Sprite[] _walkFrames;
    private int _frameIndex;
    private float _timer;
    private int _hp;

    private void Awake()
    {
        _hp = maxHp;
        _walkFrames = new Sprite[walkFrameNames.Length];
        for (int i = 0; i < walkFrameNames.Length; i++)

```



```

    {
        _walkFrames[i] = atlas.GetSprite(walkFrameNames[i]);
    }

    if (hitbox != null)
        hitbox.enabled = false;

    RefreshUI();
}

private void Update()
{
    AnimateWalk();
}

private void AnimateWalk()
{
    if (_walkFrames == null || _walkFrames.Length == 0) return;

    _timer += Time.deltaTime;
    float frameDuration = walkFrameDurations[Mathf.Clamp(_frameIndex, 0,
walkFrameDurations.Length - 1)];

    if (_timer >= frameDuration)
    {
        _timer -= frameDuration;
        _frameIndex = (_frameIndex + 1) % _walkFrames.Length;
        spriteRenderer.sprite = _walkFrames[_frameIndex];
    }
}

public void EnableAttackHitboxForWindow(float seconds)
{
    if (hitbox == null) return;
    StopAllCoroutines();
    StartCoroutine(EnableHitboxRoutine(seconds));
}

private System.Collections.IEnumerator EnableHitboxRoutine(float seconds)
{
    hitbox.enabled = true;
    yield return new WaitForSeconds(seconds);
    hitbox.enabled = false;
}

public void ApplyDamage(int amount)
{
    _hp = Mathf.Max(0, _hp - amount);
}

```

```

        RefreshUI();
        if (_hp == 0)
        {
            // play death clip, disable locomotion, etc.
        }
    }

    private void RefreshUI()
    {
        if (healthFill != null)
            healthFill.fillAmount = (float)_hp / maxHp;
    }
}

public class EldoriaHitbox : MonoBehaviour
{
    [SerializeField] private int damage = 10;

    private void OnTriggerEnter2D(Collider2D other)
    {
        var target = other.GetComponentInParent<EldoriaActor2D>();
        if (target != null)
            target.ApplyDamage(damage);
    }
}

```

This Unity example follows the documented engine model: **SpriteAtlas** for batching-friendly texture organization, **SpriteRenderer** sorting for 2D draw order, **trigger colliders** for hit windows, and an **Image** on a Canvas for HUD integration. Unity’s documentation on Sprite Atlas, sorting, trigger behavior, and CanvasScaler makes this a natural implementation of the standards above. ⁵⁷

If a Godot implementation is added later, the closest equivalents are **AnimatedSprite2D** or **AnimationPlayer** for animation, **CollisionObject2D layers/masks** for combat interaction, **CanvasLayer + Control** for screen-space UI, **NinePatchRect** for scalable panels, **TextureAtlas** for imported sheets, and **GPUParticles2D** for VFX. ⁵⁸

Implementation Checklists and Reference Structure

A guide like this only stays authoritative if it is enforced. The best approach is to treat validation as part of the pipeline, not as a late art review step. A practical enforcement checklist is below.

Checkpoint	Required pass condition
Naming	Asset ID matches the canonical pattern and folder/domain placement is correct
Visual style	Palette family assigned; light direction correct; silhouette readable at 1× and 3× scale

Checkpoint	Required pass condition
Technical metadata	Canvas size, footprint, pivot, palette, loop flag, and tags exist
Animation	Clip has explicit timing; non-looping clips marked; attack clips define hitbox windows
Import	Point filtering for gameplay sprites; no unnecessary Read/Write; correct sprite slicing; correct atlas family
Collision	Hurtbox always present for combatants; hitboxes transient; layers and masks match standard
UI	Panels use 9-slice or equivalent; anchors or Canvas layout respond correctly to aspect ratio changes
Performance	Scene remains under atlas, draw-call, and texture-memory targets for the chosen platform tier

Each of those checks corresponds directly to documented engine capabilities or costs: import settings, layer sorting, atlas behavior, UI scaling, collision matrices, and texture memory/compression rules are all already first-class concepts in Unity and Godot. ⁵⁹

The minimum “authoritative reference” package that Codex or Claude should receive for any new asset request is:

1. asset category
2. asset ID
3. canvas size
4. footprint size
5. pivot point
6. palette family
7. directional set required
8. animation tags and frame counts
9. layering rules
10. collision metadata
11. atlas family
12. UI or gameplay usage notes

That is the shortest viable set that still maps cleanly to Aseprite tags/slices, Unity import settings and atlases, and Godot import/canvas/collision systems. Anything less forces the model to improvise. ⁶⁰

A good final convention is to separate **must-have** from **nice-to-have** generation rules for AI systems:

- MUST:
- use canonical names

- stay on master palette or declared sub-palette
- respect canvas, pivot, and footprint
- output per-frame timing
- define hitbox/hurtbox windows for combat clips
- export to PNG + JSON metadata

SHOULD:

- reuse established material ramps
- reuse body rigs and attachment points
- pack by atlas family
- keep effect sheets in 16/32/64 cell sizes
- stay within scene budget targets

That distinction matters because it lets automated tools fail loudly on non-negotiables while still leaving room for creative iteration on optional polish. It is also the right balance for a guide that must serve both human artists and coding agents. ⁶¹

The most important unresolved variables — because they were not specified — are **the exact Eldoria-V2 art style, whether the game is strictly pixel art or mixed-resolution 2D, whether combat needs 4-direction or 8-direction attack coverage, whether portraits are pixel or painted, and the final low-end mobile device target**. Until those are fixed, the recommendations above should be treated as the default contract: **pixel-art RPG, 640×360 world baseline, 16×16 grid grammar, 32×48 human actors, PNG runtime exports, .aseprite source, atlas-based runtime loading, and mobile-first texture/draw-call limits**. That is the most stable, engine-supported, and AI-friendly starting point identified in the source material. ⁶²

¹ ⁶ ⁶² Multiple resolutions — Godot Engine (stable) documentation in English

https://docs.godotengine.org/en/stable/tutorials/rendering/multiple_resolutions.html

² ²⁵ ³⁰ ⁵³ ⁵⁵ ⁵⁶ ⁶⁰ Docs - Tags

https://www.aseprite.org/docs/tags/?utm_source=chatgpt.com

³ ⁹ ¹⁰ PIXEL ART TUTORIAL: BASICS

<https://www.derekyu.com/makegames/pixelart.html>

⁴ ⁵ ²² ²³ ⁴⁸ Unity - Manual: 2D game creation workflow

<https://docs.unity3d.com/6000.4/Documentation/Manual/2d-game-creation-workflow.html>

⁷ Pixelblog - 55 - Top Down Character Animation — SLYNYRD

<https://www.slynyrd.com/blog/2025/3/24/pixelblog-55-top-down-character-animation>

⁸ ⁴⁶ Canvas Scaler | Unity UI | 1.0.0

<https://docs.unity3d.com/Packages/com.unity.ugui%401.0/manual/script-CanvasScaler.html>

¹¹ ²¹ ⁴⁴ PIXEL ART: COMMON MISTAKES

<https://www.derekyu.com/makegames/pixelart2.html>

¹² ²⁶ ²⁷ ⁵⁹ Unity - Manual: Sprite (2D and UI) texture Import Settings window reference

<https://docs.unity3d.com/6000.4/Documentation/Manual/texture-type-sprite.html>

13 18 54 **Introduction to Aseprite Importer | 2D Aseprite Importer | 1.0.1**

<https://docs.unity3d.com/Packages/com.unity.2d.aseprite%401.0/manual/index.html>

14 15 29 **Importing images — Godot Engine (stable) documentation in English**

https://docs.godotengine.org/en/stable/tutorials/assets_pipeline/importing_images.html

16 50 **Unity - Manual: Work with vector graphics**

<https://docs.unity3d.com/6000.4/Documentation/Manual/ui-systems/work-with-vector-graphics.html>

17 **Manual: 2D feature set**

https://docs.unity3d.com/6000.3/Documentation/Manual/2DFeature.html?utm_source=chatgpt.com

19 **Docs - Color-mode**

https://www.aseprite.org/docs/color-mode/?utm_source=chatgpt.com

20 **The Color Bar and Palettes - Tutorial**

https://aseprite.org/docs/tutorial/color-bar-tutorial/?utm_source=chatgpt.com

24 61 **Frequently asked questions | 2D Aseprite Importer | 1.1.11**

<https://docs.unity3d.com/Packages/com.unity.2d.aseprite%401.1/manual/ImporterFAQ.html>

28 39 40 42 57 **Unity - Manual: Sprite atlases**

<https://docs.unity3d.com/6000.4/Documentation/Manual/sprite/atlas/atlas-introduction.html>

31 **Pixelblog - 8 - Intro to Animation — SLYNYRD**

<https://www.slynyrd.com/blog/2018/8/19/pixelblog-8-intro-to-animation>

32 **Pixelblog - 9 - Melee Attacks — SLYNYRD**

<https://www.slynyrd.com/blog/2018/9/8/pixelblog-9-melee-attacks>

33 **Pixelblog - 24 - Items — SLYNYRD**

<https://www.slynyrd.com/blog/2019/12/21/pixelblog-24-items>

34 **Docs - Sprite-sheet**

https://www.aseprite.org/docs/sprite-sheet/?utm_source=chatgpt.com

35 **Physics introduction - Godot Docs**

https://docs.godotengine.org/en/stable/tutorials/physics/physics_introduction.html?utm_source=chatgpt.com

36 **Change the sorting order of 2D GameObjects**

https://docs.unity3d.com/6000.0/Documentation/Manual/2d-renderer-sorting.html?utm_source=chatgpt.com

37 **Texture Sheet Animation module reference**

https://docs.unity3d.com/6000.4/Documentation/Manual/PartSysTexSheetAnimModule.html?utm_source=chatgpt.com

38 **Scripting API: TextureSheetAnimationModule**

https://docs.unity3d.com/6000.4/Documentation/ScriptReference/ParticleSystem.TextureSheetAnimationModule.html?utm_source=chatgpt.com

41 **Unity - Manual: Sprite Atlas properties reference**

<https://docs.unity3d.com/2022.3/Documentation/Manual/class-SpriteAtlas.html>

43 **GPUParticles2D — Godot Engine (stable) documentation in English**

https://docs.godotengine.org/en/stable/classes/class_gpunparticles2d.html

45 **What's new in version 1.0.0 | 2D Aseprite Importer | 1.0.1**

<https://docs.unity3d.com/Packages/com.unity.2d.aseprite%401.0/manual/whats-new.html>

47 9-slicing

https://docs.unity3d.com/6000.4/Documentation/Manual/sprite/9-slice/9-slicing.html?utm_source=chatgpt.com

49 Canvas layers — Godot Engine (latest) documentation in English

https://docs.godotengine.org/en/latest/tutorials/2d/canvas_layers.html

51 Packing sprites into atlas textures

https://docs.unity3d.com/6000.4/Documentation/Manual/sprite/atlas/atlas-landing.html?utm_source=chatgpt.com

52 Docs - Exporting

https://www.aseprite.org/docs/exporting/?utm_source=chatgpt.com

58 2D sprite animation - Godot Docs

https://docs.godotengine.org/en/stable/tutorials/2d/2d_sprite_animation.html?utm_source=chatgpt.com